Java Silver（SE 17・1Z0-825-JPN）　4 クラスの定義とインスタンス　#10

パターンマッチングとレコード

調べてキャストして変数に入れるを 1 回で。値をまとめるクラスを 1 行で

各ページの下に、根拠にした Oracle の資料（JLS 6.3・8.10・14.30・15.20.2、Java SE 17 API Record）を書いた（青い文字を押すと開く）。コードは Claude が作り、JDK 17 で動かして確かめた

全体像 試験内容チェックリスト 4（instanceof のパターン・マッチング、レコード）

1 instanceof のパターン
o instanceof String s。null は false。いつも true の組み合わせはエラー

2 パターン変数のスコープ
確実にマッチした場所だけ。&& の右は OK、|| の右は NG

3 レコード
record Point(int x, int y)。final で、アクセサは x()

4 レコードのコンストラクタ
正規・コンパクト・正規でない（this(…) 必須）

機能	Java 17 では
instanceof のパターン	正式（Java 16 から）
レコード	正式（Java 16 から）
switch のパターン	プレビュー（ふつうはコンパイルエラー）
レコード・パターン o instanceof Point(int x, int y)	まだない（Java 21 から）

試験は Java 17 の範囲。新しい Java の書き方は「コンパイルエラーになる」側の選択肢として出ることがある。

出典：Oracle「Java SE 17 Programmer I（1Z0-825-JPN）」試験内容チェックリスト／JEP 394・JEP 395（Java 16 で正式）・JEP 406（Java 17 のプレビュー）／結果は JDK 17 で確かめたもの

第1章

instanceof のパターン・マッチング

調べる・キャストする・変数に入れる、を 1 つの式で。

キャストからパターンへ

前の書き方（Claude が作成）

```
if (o instanceof String) {
  String s = (String) o;      // 自分でキャスト
  System.out.println(s.length());
}
```

パターン・マッチング

```
if (o instanceof String s) {   // 型パターン
  System.out.println(s.length());
}
```



マッチしたとき（true のとき）だけ ②③ が起きる

コード（o は Object 型）	結果（JDK 17）
o が "hello"：`o instanceof String s`	true、s は "hello"
o が 42：`o instanceof Integer n && n > 40`	true
o が 42：`o instanceof Number n`	true（Integer は Number の一種）
o が null：`o instanceof String s`	false（例外なし）
o が null：`o instanceof Object`	false

パターン変数はふつうのローカル変数。Java 17 では final ではないので代入できる（`s = s + "!"`）。final を付けると代入できない。

出典：JLS 15.20.2 The instanceof Operator（If the value of the RelationalExpression is the null reference, then the result is false／A side effect of a true result is that pattern variables declared in Pattern will be initialized）／JLS 14.30.1／結果は JDK 17 で確かめたもの

書けるパターン・書けないパターン

式	結果（JDK 17）
`o instanceof Integer i`（o は Object）	OK
`o instanceof final String t`	OK（t は代入不可）
`o instanceof int i`	unexpected type
`o instanceof var v`	'var' is not allowed here
`str instanceof String s`（str は String）	expression type String is a subtype of pattern type String
`str instanceof String`（パターンなし）	OK（true）
`i instanceof String s`（i は Integer）	incompatible types: Integer cannot be converted to String

いつも true はエラー：左の型がパターンの型と同じか、その部分型なら、調べる意味がないのでコンパイルエラー。パターンのない型の比較ならエラーにならない。

ありえない組み合わせもエラー：キャストできない型どうし（Integer と String）は、型の比較でもパターンでもコンパイルエラー。

パターンの型は参照型だけ。var・基本型・初期化子（= 値）は書けない。

出典：JLS 15.20.2（If the type of the RelationalExpression is a subtype of the type of the Pattern, then a compile-time error occurs／The RelationalExpression must be compatible with the Pattern）／JLS 14.30.1（The LocalVariableType denotes a reference type (and furthermore is not var)／The VariableDeclarator has no initializer）／結果は JDK 17 で確かめたもの

第2章

パターン変数のスコープ

使えるのは「確実にマッチした」と言える場所だけ。

if・&&・||・? :

s を使える場所（Claude が作成）

```
if (o instanceof String s) {
    s.length() ← OK
} else {
    s ← cannot find symbol
}
```

o instanceof String s && **s.isEmpty()**	右は OK
o instanceof String s || s.isEmpty()	右は NG
o instanceof Integer i ? i + 1 : 0	true 側は OK

コード	結果（JDK 17）
"hello" で `s.length() > 3`（&&）	long hello
"hi" で同じ条件、else あり	short（else へ）
f(5)・f("5")（? : で i + 1）	int 6 / other
外に String s があるのに `o instanceof String s`	variable s is already defined
if (…) { } のあとで s	cannot find symbol

理由：&& の右は左が true のときだけ評価される。|| の右は左が false（マッチしなかった）ときに評価されるので、s に値がない。

出典：JLS 6.3.1.1（A pattern variable introduced by a when true is definitely matched at b）・6.3.1.2（A pattern variable introduced by a when false is definitely matched at b）・6.3.1.4・6.3.2.2／結果は JDK 17 で確かめたもの

! と早期 return・while

```java
static void m(Object o) {
  if (!(o instanceof String s)) {
    System.out.println("not string");
    return;                 // ここで抜ける
  }
  System.out.println(s.length());   // OK
}
m("abc");  m(10);
```

```
3
not string
```

return がないと、マッチしなかったときも if のあとに来るので s は使えない（cannot find symbol）。

```java
Object o = "abc";
while (!(o instanceof Integer i)) {
  o = ((String) o).length();
}
System.out.println(i * 2);     // 6
```

if (!(… s)) S のあとで s を使うには	S の中身
使える	return・throw などで必ず抜ける
使えない	表示するだけ（正常に終わる）

while も同じ考え方。条件が false で抜けるのはマッチしたときだけ（break がない）なら、ループのあとで使える。

出典：JLS 6.3.2.2（A pattern variable is introduced by if (e) S iff (i) it is introduced by e when false and (ii) S cannot complete normally）・6.3.2.3（S does not contain a reachable break statement whose break target contains S）・6.3.1.3／結果は JDK 17 で確かめたもの

第3章

レコード

1 行で、フィールド・アクセサ・コンストラクタ・equals・hashCode・toString。

1 行で作られるもの

```
record Point(int x, int y) { }
```

ほぼ次のクラスと同じ意味（Claude が書き直した）

```
final class Point extends Record {
  private final int x;
  private final int y;
  Point(int x, int y) { this.x = x; this.y = y; }
  public int x() { return x; }
  public int y() { return y; }
  // equals・hashCode・toString も全コンポーネントから
}
```

実際には extends Record とは書けない（レコードは extends を書けない）。説明のための形。

```
Point p = new Point(1, 2);
Point q = new Point(1, 2);
System.out.println(p.x() + " " + p + " " + p.equals(q) + " " + (p == q));
```

```
1 Point[x=1, y=2] true false
```

外のクラスから	結果（JDK 17）
`p.x()`	1
`p.getX()`	cannot find symbol
`p.x`	x has private access in Point
`new Point()`	constructor Point in record Point cannot be applied to given types
`new E(){record E() { }}`	E[]

出典：JLS 8.10.1（a private field declared implicitly, and a public accessor method）・8.10.3（A component field is private, final, and non-static）・8.10.4（a record class does not implicitly declare a default constructor）／API Record／結果は JDK 17 で確かめたもの

equals・toString とアクセサの書き直し

メソッド	自動で作られる動き
equals	同じレコード型で、全コンポーネントが等しければ true
hashCode	equals が true どうしなら同じ値
toString	型名・コンポーネント名・値を含む。JDK 17 では `Person[name=Ann, age=3]`

API は toString の正確な形を決めていない（変わることがある）。試験の選択肢では JDK の表示どおりの形が使われる。

> **自分で書けばそちらが使われる。**`public boolean equals(Object o) { return true; }` と書けば、new P(1).equals(new P(2)) も true。

```
record P(int x) {
  public int x() { return x * 10; }   // 書き直し
}
P p = new P(2);
System.out.println(p.x() + " " + p);
```

```
20 P[x=2]
```

書き直すとき	結果（JDK 17）
`public int x()`	OK
`int x()`（public なし）	invalid accessor method in record P

toString はフィールドの値から作られた（x=2）。アクセサを書き直しても toString は変わらなかった（JDK 17）。

出典：API Record（The implicit declaration of the Object.equals(Object), Object.hashCode(), and Object.toString() methods are derived from all of the component fields／The precise format produced by this implicitly provided implementation is subject to change）／JLS 8.10.3（The accessor method must be a public instance method with no formal parameters）／結果は JDK 17 で確かめたもの

3 種類のコンストラクタ

① 正規コンストラクタ（ふつうの形）

```
record P(int x, int y) {
  P(int x, int y) { this.x = x; this.y = y * 2; }
}   // new P(1, 2) → P[x=1, y=4]
```

② コンパクト・コンストラクタ（かっこなし）

```
record Name(String s) {
  Name { s = s.trim(); }        // s は引数
}   // new Name("  ann ").s() → "ann"
```

③ 正規でないコンストラクタ（this(…) が必須）

```
record P(int x, int y) {
  P(int x) { this(x, 0); }
}   // new P(7) → P[x=7, y=0]
```

書き方	結果（JDK 17）
コンパクトで `this.s = s.trim();`	cannot assign a value to final variable s
コンパクトで `return;`	invalid compact constructor
コンパクトと `P(int x)` を両方	constructor P(int) is already defined
`P(int x) { }`（this(…) なし）	constructor is not canonical, so its first statement must invoke another constructor
コンパクトで `x = x + 1;`（new P(1)）	x() は 2
コンパクトで v < 0 なら throw	new Age(-1) で例外

コンパクト・コンストラクタの中の名前は引数。本体が終わると、引数の値でフィールドが自動で初期化される。

出典：JLS 8.10.4（The body of every non-canonical constructor in a record declaration must start with an alternate constructor invocation）・8.10.4.2（The body must not contain a return statement／must not contain an assignment to a component field／the simple name c … denotes the implicit formal parameter）／結果は JDK 17 で確かめたもの

レコードに書けるもの・書けないもの

書いたもの	結果（JDK 17）
`static int count;`・static メソッド	OK
インスタンスメソッド `int twice()`	OK
`implements Comparable<P>`	OK
`int y;`（static なし）	field declaration must be static
`{ … }`（インスタンス初期化子）	instance initializers not allowed in records
`extends Object`	'{' expected
`abstract record`	modifier abstract not allowed here
`class Q extends P`	cannot inherit from final P

浅い不変（Claude が作成）

```
record R(List<String> list) { }
R r = new R(new ArrayList<>());
r.list().add("a");                // できてしまう
System.out.println(r);           // R[list=[a]]
```

> **final なのはフィールドだけ。**指しているリストの中身は変えられる。守りたいならコンパクト・コンストラクタで `list = List.copyOf(list);` のようにコピーする（#09 の防御的コピー）。

API Record の説明も、正規コンストラクタを書く理由に「引数の検証」「変えられるコンポーネントの防御的コピー」を挙げている。

出典：JLS 8.10（A record class is implicitly final／It is a compile-time error if a record declaration has the modifier abstract／A record declaration does not have an extends clause）・8.10.2（non-static field declaration／instance initializer）／API Record（validate constructor arguments, perform defensive copies on mutable components）／結果は JDK 17 で確かめたもの

問われ方とまちがえやすい点

こう聞かれたら	答えの考え方
instanceof の結果	null は false。部分型なら true。パターンが付くと、いつも true の組み合わせはコンパイルエラー
パターン変数を使えるか	確実にマッチした場所だけ。else・|| の右・if のあと（return なし）は NG
Java 17 で書けるか	instanceof のパターンとレコードは OK。switch のパターン、レコード・パターンは NG
レコードの表示・比較	toString は Point[x=1, y=2]、equals は中身、== は別オブジェクトなら false
レコードがコンパイルできるか	インスタンスフィールド・初期化子・extends・abstract は NG。アクセサは public。正規でないコンストラクタは this(…)
コンパクト・コンストラクタの結果	中の名前は引数。引数を変えればフィールドに入る。this.x への代入は NG

出典：このPDFの各ページの資料

まとめ

instanceof のパターン
o instanceof String s。null は false

スコープ
マッチが確実な場所だけ。&& は OK、|| は NG

レコード
final・private final フィールド・x()・自動の equals/toString

コンストラクタ
正規・コンパクト（引数を整える）・this(…)

こんなとき	結果
`null instanceof String`	false
else の中でパターン変数	コンパイルエラー
record の `p.getX()`	コンパイルエラー
record に `new P()`（コンポーネントあり）	コンパイルエラー
同じ値のレコードを equals	true

次は #11 継承とポリモーフィズム。extends・super・オーバーライド・抽象クラス・キャストと final へ進む。

出典：このPDFの各ページの資料